Importer Developer’s Handbook

Document: Importer Developer’s Handbook

Document Version: 1.0
Document Date: 1 Nov 26
Applies to: Photo Manager Version 2.0.0
Project JSON Schema: Version 2

Table of Contents

1. Introduction....4
    1.1. Purpose and Scope....4
    1.2. What Is a Standalone Importer?....4
    1.3. Potential Source Systems....4
        1.3.1. Personal Photo Libraries and Collections....4
        Travel, Cycling and Outdoor Activity Platforms....5
        Social Networks and Photo-Sharing Services....5
        Travel Journals and Online Diaries....5
        Cloud Storage and File-Sharing Services....6
        Camera and Mobile-Device Ecosystems....6
        Public and Institutional Photo Collections....6
        Assessing a Potential Source....7
        1.3.2. Choosing an Importer Project....7
    1.4. Importer Responsibilities and Boundaries....8
    1.5. Related Specifications and Documentation....8
    1.6. Requirements, Recommendations and Examples....8
2. Photo Manager Workspace....9
    2.1. Understanding the Workspace....9
    2.2. Determining the Workspace Location....9
    2.3. Platform Independence and Filesystem Paths....10
3. Project Structure and File Management....10
    3.1. Where Projects Belong....10
    3.2. Required Project Directory Structure....11
        3.2.1. Resolving Photograph Paths....12
        3.2.2. Authoritative Requirements....13
    3.3. Photo Files and Supporting Files....13
        3.3.1. Photograph Files and Photo Records....13
        3.3.2. Metadata and Supporting Files....13
        3.3.3. Final Integrity Check....14
    3.4. Temporary Files and Incomplete Projects....14
        3.4.1. Staging Location....14
        3.4.2. Temporary Project File....14
        3.4.3. Validation Before Publication....15
        3.4.4. Publishing the Completed Project....15
        3.4.5. After Publication....15
4. The Role of project.json....16
    4.1. The Role of project.json....16
    4.2. Following the Project JSON Reference....17

4.3. Creating Photo Records....17
4.4. File Names and Duplicate Records....18
5. Completing and Publishing a Project....19
5.1. The Project Creation Lifecycle....19
5.2. Keeping Incomplete Projects Undiscoverable....19
5.3. Validating Before Publication....20
5.4. Making a Completed Project Discoverable....21
5.5. Protecting Existing and Published Projects....21
6. Validation and Testing....22
6.1. Structural and JSON Validation....22
6.1.1. JSON and Project Structure....22
Completion Status....23
6.1.2. Validation Outcome....23
6.2. Checking File References....23
6.2.1. Filename Collisions....24
6.3. Verifying Metadata and Photo Records....24
Metadata Accuracy and Preservation....24
6.3.1. Identifiers and Duplicate Records....24
6.4. Testing....25
6.4.1. Project Discovery and Selection....25
6.4.2. Project Loading and Photograph Access....25
6.4.3. Metadata Propagation and Initial Values....25
6.4.4. Portability and Repeatability....26
6.4.5. Recording Test Results....26
7. Development Guidelines and Recommendations....26
7.1. Language and Technology Independence....26
7.2. Cross-Platform Development....27
7.3. Dependencies and Configuration....27
7.3.1. Dependencies....28
7.3.2. Configuration....28
7.3.3. Installation and Operation....28
7.4. Separation of Responsibilities....28
7.5. Logging, Diagnostics and Error Reporting....29
7.5.1. Logging....29
7.5.2. Error Handling....29
7.5.3. Diagnostic Information and Recovery....30
7.6. Repeatability and Data Integrity....30
7.6.1. Repeatable Imports....30
7.6.2. Protecting Data Integrity....30
7.6.3. Recovery and Cleanup....31
7.7. Documentation and Maintainability....31
7.7.1. Documentation....31
7.7.2. Maintainability....31
7.7.3. Licensing....32
8. Practical Examples....32
8.1. Staging the Import and Creating importing.json....32
8.1.1. Creating the Initial Checkpoint....32
8.1.2. Example importing.json....33
8.1.3. The Importer-Specific Data....34
8.1.4. Why Use importing.json?....34
8.2. Acquiring and Validating Files and Constructing Photo Records....35
8.2.1. Processing one photograph....35

8.2.2. Copying and testing the photograph....35
8.2.3. Gathering information for the photo record....36
8.2.4. Proposing metadata values....36
8.2.5. Understanding the grouping fields....38
8.2.6. Constructing the photo record....39
8.2.7. Handling a candidate that fails testing....40
8.2.8. Appending the record and saving progress....40
8.3. Preparing and Validating the Completed Project....41
8.3.1. Preparing the project data....41
8.3.2. Resulting project structure....42
8.3.3. Validating the prepared project....43
8.3.4. Keeping the working and prepared files separate....43
8.4. Publishing the Completed Project....44
8.4.1. Choosing the final project directory....44
8.4.2. Finalising the Filesystem Importer project....45
8.4.3. Confirming publication....45
8.4.4. After publication....45
8.5. Lessons from the Filesystem Importer....46
8.5.1. Keep acquisition separate from publication....46
8.5.2. Preserve enough information to recover....46
8.5.3. Do not modify the original source unnecessarily....46
8.5.4. Treat metadata as proposals, not unquestionable facts....47
8.5.5. Validate acquired files before recording them as successful....47
8.5.6. Make progress and failure visible....47
8.5.7. Keep Importer-specific information out of the published project....47
8.5.8. Publish only after the project is ready....48
8.5.9. Final observation....48
9. Troubleshooting Photo Manager Compatibility....48
9.1. Project is not discovered by Photo Manager....48
9.2. Project is discovered but cannot be opened....49
9.3. Project opens but photographs are missing....49
9.4. Project opens but metadata is missing or unexpected....49
9.5. Final check....50
10. Reference....50
10.1. Relevant Specifications....50
10.1.1. Photo Manager Project JSON Reference....50
Photo Manager Core Design Specification....50
Source System Documentation....51
10.1.2. Using the Specifications....51
10.2. Glossary....51
10.3. Developer Checklist....53
10.3.1. Project Format and Compatibility....54
Workspace and File Management....54
Acquisition and Metadata....54
Validation and Publication....54
Testing and Release....55

1. Introduction

1.1. Purpose and Scope

This Handbook provides practical guidance for developers creating standalone Importers for Photo Manager. It focuses on producing valid Photo Manager projects that can be discovered, loaded and used by the application.

The Handbook covers project structure, the generation and validation of `project.json`, file management, project publication, portability, testing and recommended development practices.

It does not prescribe how an Importer must communicate with a source system, authenticate, enumerate content or acquire assets. These matters depend on the source system and are the responsibility of the Importer developer.

1.2. What Is a Standalone Importer?

A standalone Importer is an application that obtains photographs and associated information from a source and prepares them as a Photo Manager project.

An Importer is responsible for interacting with its source, acquiring the required assets, organising the resulting files, and generating a project that conforms to the Photo Manager project format.

Importers operate independently of Photo Manager. They do not use a runtime Core–Importer API or shared Importer SDK. Their integration with Photo Manager is through the project they produce, including its `project.json` file and associated assets.

1.3. Potential Source Systems

A standalone Importer can obtain photographs and associated information from a wide range of sources. The source does not have to be an online service. It may be a local collection of files, an exported photo library, an online platform or an institutional archive.

The following categories provide examples of potential sources that developers might consider. They are illustrative rather than exhaustive, and inclusion does not imply that a particular service provides an API or permits unrestricted access to its content.

1.3.1. Personal Photo Libraries and Collections

These services and applications organise photographs taken by individuals or collected from different devices.

Examples include:

- **Apple Photos** – Photo libraries maintained on iPhone, iPad and Mac.
- **iCloud Photos** – Photographs and videos synchronised through Apple's cloud service.

- **Google Photos** – Cloud-based photo libraries, albums and associated information.
- **Microsoft Photos and OneDrive** – Local and cloud-based photograph collections.
- **Adobe Lightroom** – Organised photo libraries, albums, descriptions and editing information.
- **digiKam** – Locally managed photo collections with extensive metadata and tagging capabilities.

Travel, Cycling and Outdoor Activity Platforms

These platforms record journeys, activities and routes. Some also associate photographs with particular activities or locations.

Examples include:

- **komoot** – Cycling, walking and other outdoor activities, including tours, routes and associated photographs.
- **Strava** – Cycling, running and other recorded activities, GPS tracks, statistics and photographs attached to activities.
- **Ride with GPS** – Cycling routes, recorded rides, route information and associated photographs.
- **Wikiloc** – Outdoor trails, GPS tracks, waypoints and photographs associated with activities.
- **Garmin Connect** – Recorded activities, GPS tracks, locations and associated device information.
- **Polar Flow** – Recorded sporting activities, routes and exercise information.

Social Networks and Photo-Sharing Services

These platforms contain photographs accompanied by captions, comments, dates, tags and information about the people or events associated with them.

Examples include:

- **Facebook** – Personal photographs, albums, posts, captions and associated discussions.
- **Instagram** – Photographs, posts, captions, dates, hashtags and account information.
- **Flickr** – Photo collections, albums, descriptions, tags, geographic information and EXIF metadata.
- **VSCO** – Photographs, edited images, collections and profile content.
- **500px** – Photographic portfolios, image descriptions, categories and location information.
- **SmugMug** – Hosted photo collections, galleries, captions and associated information.

Travel Journals and Online Diaries

These services combine photographs with personal accounts of journeys, events and experiences.

Examples include:

- **Polarsteps** – Travel journeys, mapped routes, photographs and travel notes.
- **FindPenguins** – Travel diaries, location records, photographs and written entries.
- **WordPress** – Travel blogs and personal websites containing photographs and associated articles.
- **Blogger** – Online journals and blogs containing photographs, captions and dated entries.
- **Journiapp** – Online journals and blogs containing photographs, captions and dated entries with locations.

Cloud Storage and File-Sharing Services

These services may contain photographs and supporting information in files, directories or shared collections.

Examples include:

- **Dropbox** – Stored photographs, directory structures and shared collections.
- **Google Drive** – Photograph collections, exported albums and supporting documents.
- **Microsoft OneDrive** – Personal photograph collections and synchronised files.
- **iCloud Drive** – Photographs and other files stored within Apple's cloud storage.
- **Network Attached Storage (NAS)** – Locally managed photo collections, including systems using Synology Photos or similar applications.

Camera and Mobile-Device Ecosystems

Some manufacturers provide applications and services that organise, transfer or supplement photographs obtained from their cameras and devices.

Examples include:

- **GoPro Quik** – Photographs and videos associated with GoPro cameras and activities.
- **DJI Mimo** – Photographs and videos associated with DJI devices.
- **Canon Camera Connect** – Photographs transferred from supported Canon cameras.
- **Nikon SnapBridge** – Photographs and associated information transferred from supported Nikon cameras.
- **Mobile-device exports** – Photographs and videos exported directly from smartphones, tablets and camera memory cards.

Public and Institutional Photo Collections

Not all potential Importer sources are personal photo libraries. Publicly accessible collections may contain historical, documentary or specialist photographs accompanied by useful descriptive information.

Examples include:

- **Trove** – Digitised historical collections, including photographs and associated catalogue records.
- **Europeana** – Cultural heritage collections containing photographs and descriptive metadata.
- **Wikimedia Commons** – Freely accessible media collections with descriptions, categories and licensing information.
- **National and state archives** – Historical photographs, catalogue records, dates and location descriptions.
- **Museum and library collections** – Digitised photographs and associated collection information.

Assessing a Potential Source

Before developing an Importer for a particular source, the developer should investigate how photographs and their associated information can be obtained.

Some important considerations include:

- **Photograph availability:** Can the original photograph files be obtained, or is access limited to reduced-resolution previews or thumbnails?
- **Associated information:** What contextual information accompanies the photographs? This might include capture dates, geographic locations, descriptions, captions, comments or grouping information.
- **Asset identification:** Can the source provide a reliable way to associate each photograph with its corresponding information?
- **Access methods:** Does the source provide an API, bulk export facility, downloadable archive or another suitable means of obtaining its content?
- **Access restrictions:** Are authentication, permissions, privacy, licensing or usage restrictions involved?
- **Reliability:** Can the acquisition process handle interrupted connections, unavailable files and other failures? Can it be repeated safely when necessary?

The availability of an API or export facility must not be assumed. Access methods, service capabilities and usage restrictions vary between providers and may change over time.

1.3.2. Choosing an Importer Project

The usefulness of a source is not determined solely by the number of photographs it contains. The associated contextual information may be just as valuable.

An Importer that can preserve photographs together with their dates, locations, descriptions and relationships to other photographs can produce a much more useful Photo Manager project than one that obtains image files alone.

Developers should therefore consider both the photographs available from a source and the additional information that can be acquired and retained with them.

The choice of source, acquisition method and internal implementation is entirely the responsibility of the Importer developer, provided the resulting project conforms to the applicable Photo Manager requirements.

1.4. Importer Responsibilities and Boundaries

An Importer is responsible for producing a complete, valid project in the appropriate Photo Manager workspace.

In particular, it must ensure that:

- The project follows the required directory structure.
- The `project.json` file conforms to the applicable Project JSON Specification.
- Every photo record refers to an existing, usable file included in the project.
- Relevant metadata is preserved and represented correctly.
- An incomplete project is not discoverable by Photo Manager while it is being created.
- Once a project becomes discoverable by Photo Manager, the Importer does not modify it.

The Importer developer may choose the programming language, libraries, architecture and source-acquisition methods, provided the resulting project satisfies the applicable requirements.

1.5. Related Specifications and Documentation

The **Project JSON Reference** is the authoritative specification for the structure, fields, data types and constraints of `project.json`. The Handbook explains how to apply that specification when developing an Importer; it does not replace or override it.

The Photo Manager design documentation defines the application's project discovery, validation and workspace behaviour. Where implementation details affect how an Importer must operate, developers should consult the applicable design documentation and verify the documented behaviour against the supported application version.

Practical examples in this Handbook illustrate possible implementations. They are not additional specifications unless explicitly identified as requirements.

1.6. Requirements, Recommendations and Examples

This Handbook distinguishes between mandatory requirements and recommended practices.

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT** and **MAY**, where used in their normative sense, indicate the level of obligation associated with a statement.

- **MUST** and **MUST NOT** identify requirements that must be followed.
- **SHOULD** and **SHOULD NOT** identify recommendations that may be departed from when there is a sound reason.
- **MAY** identifies an optional approach or capability.

Examples illustrate particular techniques and expected outcomes. Unless explicitly stated otherwise, an example is illustrative rather than a requirement to use the same implementation.

2. Photo Manager Workspace

2.1. Understanding the Workspace

Photo Manager uses a designated workspace to organise its projects. An Importer must place completed projects in the appropriate location so that Photo Manager can discover and load them.

The workspace location is specified in Photo Manager's `settings.json` configuration file. The location of this configuration file depends on the user's environment and operating system.

For example, on the author's Ubuntu laptop, the configuration file is located in:

`/home/geoff/.config/photo_manager/settings.json`

This is an example of a user-specific configuration location, not a path that Importers should assume will apply to other installations.

The workspace directory is distinct from any temporary directory an Importer uses while acquiring assets or constructing a project. Importers must ensure that projects under construction are not discoverable by Photo Manager.

2.2. Determining the Workspace Location

Photo Manager uses the Python `platformdirs` library to determine the platform-specific location for its application configuration files. This allows the application to follow the conventions of the operating system rather than assume a fixed configuration directory.

In Photo Manager, the configuration directory is determined using the following Python call:

```
from pathlib import Path
from platformdirs import user_config_dir

config_directory = Path(user_config_dir("photo_manager"))
settings_file = config_directory / "settings.json"
```

The application identifier is `photo_manager`, and the settings filename is `settings.json`.

The example illustrates the mechanism used by Photo Manager. Importer developers working in other programming languages should determine how to obtain the equivalent per-user application configuration directory on their target operating systems.

Once the configuration file has been located, the Importer must read the `workspace_directory` value from `settings.json`. This value identifies the configured Photo Manager workspace in which completed projects are to be placed.

The Importer should not assume that the configuration directory or workspace path is identical across operating systems or users. It should handle missing configuration files, invalid settings and inaccessible workspace directories explicitly, reporting the problem rather than silently substituting an assumed location.

2.3. Platform Independence and Filesystem Paths

Photo Manager is designed to make porting to alternative operating systems as straightforward as possible. Importers should follow the same principle.

An Importer should avoid unnecessary assumptions about operating-system conventions, directory layouts, path separators and filesystem behaviour. The objective is to make the Importer portable without requiring major changes to its core logic when it is run on another supported platform.

The following practices are recommended:

- **Use appropriate path-handling facilities.** Use the filesystem and path-handling facilities provided by the chosen programming language or its established libraries. Avoid constructing paths by manually concatenating strings or assuming a particular path separator.
- **Avoid hard-coded locations.** Do not assume that the user's home directory, application configuration directory or Photo Manager workspace has a fixed location.
- **Separate configuration from implementation.** Obtain environment-specific locations through configuration or platform-appropriate mechanisms rather than embedding them in source code.
- **Handle filesystem errors explicitly.** Account for missing directories, insufficient permissions, unavailable storage and failed file operations.
- **Consider filename portability.** Avoid filenames that depend on characters or conventions that may not be supported consistently across target filesystems.
- **Use relative paths within projects where the Project JSON Reference requires them.** Ensure that references to project assets remain valid when a project is moved with its directory structure intact.
- **Test on supported platforms.** Where multiple operating systems are targeted, test directory creation, file access, path resolution and project validation on each target operating system.

These are development recommendations rather than requirements to use a particular language or library. Developers remain free to choose their implementation approach, provided the resulting project conforms to the Project JSON Reference and can be correctly discovered and loaded by the applicable version of Photo Manager.

3. Project Structure and File Management

3.1. Where Projects Belong

Each completed Photo Manager project MUST reside in its own directory directly beneath the configured Photo Manager workspace.

The workspace may contain multiple projects. Each project is maintained independently within its own directory, which contains the project's `project.json` file and the associated files required

by the project. This arrangement allows Photo Manager to locate individual projects without requiring their contents to be mixed with those of other projects.

For example, a workspace might contain the following directories:

```
PhotoManagerWorkspace/
├── Army/
│   ├── project.json
│   └── pictures/
├── Europe Trip/
│   ├── project.json
│   └── photos/
├── Scrool Reunion/
│   ├── project.json
│   └── photos/
└── Importers/
    └── Journi/
        └── ...
```

In this example, `Army`, `Europe Trip` and `Tschool Reunion` are project directories. `Importers/Journi` is used by the Journi Importer while assembling a project and is not itself a completed project merely because it is located within the workspace.

Importers SHOULD keep projects under construction separate from completed projects. A staging directory may be located within the workspace, provided the Importer ensures that Photo Manager cannot discover the project as complete while it is still being assembled. The staging and publication procedure is described in Section 3.5.

The name of a project directory does not have to match the project name recorded in `project.json`. The directory name identifies the project's location in the filesystem, while the project data records the name presented within Photo Manager. Importers may choose a suitable directory name, subject to the need to avoid collisions with existing projects.

An Importer MUST place a completed project directly beneath the configured workspace when publishing it. It MUST NOT place a new project inside the directory of another project. It MUST also avoid overwriting or modifying an existing project during publication.

3.2. Required Project Directory Structure

Each Photo Manager project directory contains a `project.json` file and a directory containing the project's photographs. The names of the project directory and photograph directory are not fixed, but their relationship is defined by the project data.

A typical project has the following structure:

```
<workspace>/
└── <project directory>/
    ├── project.json
    └── photos/
        ├── image1.jpg
        ├── image2.jpg
        └── ...
```

In this example:

- `<workspace>` is the configured Photo Manager workspace.
- `<project directory>` is the directory containing the individual project.
- `project.json` contains the project's data, including its name, photograph directory and photo records.
- `photos/` contains the photograph files referenced by the project.

The photograph directory does not have to be named `photos`. Its location is specified by the `image_folder` field within the `workspace` object in `project.json`.

3.2.1. Resolving Photograph Paths

Photo Manager uses the location of `project.json` as the starting point for resolving project paths. The `image_folder` value identifies the photograph directory relative to the project directory. Each photo record's `file` value identifies a photograph by its filename relative to that photograph directory.

For example, consider this project structure:

```
Holiday/
├── project.json
└── travel_photos/
    ├── beach.jpg
    └── mountains.jpg
```

The relevant project data would include:

```
{
    "workspace": {
        "event": "Holiday",
        "image_folder": "travel_photos"
    },
    "photos": [
        {
            "file": "beach.jpg"
        },
        {
            "file": "mountains.jpg"
        }
    ]
}
```

This is an illustration of the path relationship, not a complete `project.json` example. Actual project data must contain all fields required by the Project JSON Reference.

In this example, Photo Manager resolves the photograph paths as follows:

- Project directory: the directory containing `project.json`.
- Photograph directory: the project directory combined with `workspace.image_folder`.
- Photograph file: the photograph directory combined with the photo record's `file` value.

Importers MUST follow these path conventions. They MUST NOT store machine-specific absolute filesystem paths in `project.json`. Using relative paths allows a complete project directory to be moved to another location without requiring its internal file references to be rewritten.

3.2.2. Authoritative Requirements

The Project JSON Reference defines the required project structure, fields, data types and path conventions. Importer developers MUST consult that specification when generating project data.

The examples in this Handbook illustrate how the directory structure and file references work together. They do not replace the Project JSON Reference or define additional fields and constraints.

3.3. Photo Files and Supporting Files

An Importer is responsible for ensuring that all photograph files required by the completed project are stored within the project directory structure. The project must remain self-contained, with its photograph files accessible through the paths defined by `project.json`.

3.3.1. Photograph Files and Photo Records

Each photo record in `project.json` MUST refer to a photograph file that has been successfully acquired and is present in the completed project.

An Importer MUST NOT include a photo record for a file that failed to download, could not be saved, or is otherwise absent from the completed project. If an acquisition fails, the Importer should either retry the operation where appropriate or exclude the unavailable photograph from the completed project and report the failure to the user.

Before finalising the project, the Importer SHOULD verify that every referenced photograph file exists at the expected location and can be accessed. It should also check that the recorded filename and the actual filename agree, including differences in letter case where the target filesystem distinguishes them.

Photo records MUST reference files according to the path conventions defined by the Project JSON Reference. The `workspace.image_folder` value identifies the photograph directory relative to the project directory, and each photo record's `file` value identifies the file relative to that directory. Importers MUST NOT substitute absolute machine-specific paths.

3.3.2. Metadata and Supporting Files

Photographs may be accompanied by metadata or other supporting files, depending on the source system and the requirements of the project format.

Importers SHOULD preserve relevant source metadata and populate the corresponding project fields in accordance with the Project JSON Reference. This may include information such as capture dates, locations, descriptions and other contextual information available from the source.

Where the project format or the Importer's chosen approach requires supporting files, those files MUST be placed within the appropriate project directory structure and referenced correctly where applicable. Importers SHOULD avoid introducing unnecessary files or dependencies that would prevent the project from being moved or used independently of the original source system.

The Importer should not assume that metadata embedded in a photograph is a substitute for metadata required in `project.json`, or vice versa. The requirements of the Project JSON Reference determine which information must be represented in the project data.

3.3.3. Final Integrity Check

Before publishing a completed project, the Importer SHOULD perform a final integrity check to confirm that:

- Every photo record refers to a file included in the project.
- Every referenced file can be accessed at the expected path.
- The `image_folder` and photo filenames are consistent with the actual directory structure.
- Required metadata and supporting files are present.
- No photo records remain for files that failed to be acquired.

These checks help prevent projects that appear complete but contain missing photographs or broken file references. They complement, but do not replace, validation against the Project JSON Reference.

3.4. Temporary Files and Incomplete Projects

Importers SHOULD assemble projects in a staging directory separate from the directories of completed projects. This allows photographs to be downloaded, project data to be generated and validation to be performed without exposing an incomplete project to Photo Manager.

3.4.1. Staging Location

Where practical, the staging directory SHOULD be located on the same filesystem as the configured Photo Manager workspace. This allows the completed project to be handed over by moving or renaming its directory rather than copying potentially hundreds of photographs.

The staging location must have sufficient capacity for the expected size of the import. Importers SHOULD keep temporary files and intermediate data separate from the final project contents, and remove unnecessary temporary data when the import has completed successfully.

3.4.2. Temporary Project File

While a project is being assembled, the Importer SHOULD use a temporary filename for its project data file rather than `project.json`.

The purpose of this convention is to prevent Photo Manager from discovering a project before the import is complete and the project has been validated. The temporary filename distinguishes a project under construction from one ready for discovery.

The Journi Importer uses `importing.json` as a temporary filename for this purpose. Its contents follow the applicable Project JSON Reference; the different filename does not define a different project format.

Importers MAY use another temporary filename or an alternative mechanism, provided it reliably prevents Photo Manager from discovering the project as complete while it is still being assembled.

Before publication, the Importer must ensure that the project data file has the filename required by Photo Manager, namely `project.json`, and that the project is complete and valid.

3.4.3. Validation Before Publication

Before publishing a project, the Importer MUST confirm that the project is complete and conforms to the applicable project requirements.

At a minimum, the Importer SHOULD verify that:

- The project data can be read and parsed.
- The project data conforms to the Project JSON Reference.
- Every photo record refers to a file that exists at the expected location.
- Required metadata and supporting files are present.
- No records remain for photographs that failed to be acquired.
- The project directory contains the files required for normal use by Photo Manager.

If validation fails, the Importer MUST NOT publish the project. It should report the problem and leave the incomplete project in the staging area, where it can be examined or cleaned up according to the Importer's own recovery procedures.

3.4.4. Publishing the Completed Project

Once the project has passed validation, the Importer can publish it using the following sequence:

1. Call the valid project file `project.json`.
2. Move or rename the project directory into its final location directly beneath the configured Photo Manager workspace.
3. Confirm that publication completed successfully.

The Importer MUST ensure that Photo Manager cannot discover the project as complete before the import and validation steps have finished. The publication procedure must also avoid overwriting an existing project.

Where staging and destination are on the same filesystem, moving or renaming the project directory is generally a more efficient handover than copying its contents. If the move crosses filesystem boundaries, the operation may require copying the files. In that case, the Importer MUST ensure that the destination does not become discoverable until copying and validation have completed.

3.4.5. After Publication

Once the project becomes discoverable by Photo Manager, the Importer MUST NOT modify its files or project data. Any subsequent changes to the project must be performed through the appropriate Photo Manager functionality.

If publication fails, the Importer should report the failure and avoid leaving a partially published project that could be mistaken for a completed import.

4. The Role of `project.json`

The `project.json` file is the central description of a Photo Manager project. It contains the project information and photo records that Photo Manager uses to identify the project and locate its photographs.

The file describes the project and references its photograph files; it does not replace those files. A valid project therefore requires both correctly structured project data and the files referenced by that data.

The Importer MUST create a `project.json` file that conforms to the applicable Project JSON Reference. The file MUST identify the correct project information and provide photo records whose file references correspond to the actual contents of the project directory.

The project directory's location determines the base from which the paths in `project.json` are resolved. The `workspace.image_folder` value identifies the photograph directory relative to the project directory, while each photo record's `file` value identifies a photograph relative to that directory. Importers MUST follow these path conventions rather than storing absolute paths tied to a particular computer.

The Project JSON Reference defines the required fields, their meanings, permitted values and structural constraints. Importers MUST follow that reference rather than relying on assumptions based on another project, an example, or the behaviour of a particular Importer.

A project that contains photograph files but has missing, incorrect or invalid project data may not be discoverable or load correctly in Photo Manager. Generating and validating `project.json` is therefore a central part of building a valid project, not merely a final administrative step.

4.1. The Role of `project.json`

The `project.json` file is the central description of a Photo Manager project. It contains the project information and photo records that Photo Manager uses to identify the project and locate its photographs.

The file describes the project and references its photograph files; it does not replace those files. A valid project therefore requires both correctly structured project data and the files referenced by that data.

The Importer MUST create a `project.json` file that conforms to the applicable Project JSON Reference. The file MUST identify the correct project information and provide photo records whose file references correspond to the actual contents of the project directory.

The project directory's location determines the base from which the paths in `project.json` are resolved. The `workspace.image_folder` value identifies the photograph directory relative to the project directory, while each photo record's `file` value identifies a photograph relative to that

directory. Importers MUST follow these path conventions rather than storing absolute paths tied to a particular computer.

The Project JSON Reference defines the required fields, their meanings, permitted values and structural constraints. Importers MUST follow that reference rather than relying on assumptions based on another project, an example, or the behaviour of a particular Importer.

A project that contains photograph files but has missing, incorrect or invalid project data may not be discoverable or load correctly in Photo Manager. Generating and validating `project.json` is therefore a central part of building a valid project, not merely a final administrative step.

4.2. Following the Project JSON Reference

The Project JSON Reference is the authoritative specification for the structure and contents of `project.json`. Importers MUST follow its requirements when creating project data.

The reference defines the required fields, data types, permitted values and relationships between project information and photo records. Importer developers MUST consult the reference rather than assume that fields, values or structures used in an example are sufficient for a valid project.

When constructing `project.json`, an Importer MUST:

- Include all required fields and provide values of the specified types.
- Follow the defined structure for project information and photo records.
- Use field names and permitted values exactly as specified.
- Represent file references according to the documented path conventions.
- Satisfy any constraints and relationships defined by the reference.

Examples in this Handbook are intended to illustrate particular concepts. Unless explicitly identified as complete and valid, an example MUST NOT be treated as a complete project specification or as a substitute for the Project JSON Reference.

If a requirement is unclear, the Importer developer SHOULD consult the relevant specification and verify the interpretation against a known-valid project or the behaviour of Photo Manager. An Importer SHOULD NOT introduce undocumented fields or rely on undocumented behaviour as a substitute for following the specification.

Before publishing a project, the Importer SHOULD validate `project.json` against the applicable requirements. Successful JSON parsing alone does not establish that the file conforms to the Project JSON Reference or that the project can be loaded correctly by Photo Manager.

4.3. Creating Photo Records

Photo records describe the photographs included in a Photo Manager project. Each record MUST follow the structure defined by the Project JSON Reference and contain the information required for Photo Manager to identify and use the photograph.

An Importer MAY create photo records before, during or after the acquisition of the corresponding photographs. For example, the Journi Importer creates photo records during registration, before the photographs are downloaded.

Regardless of when records are created, the Importer MUST ensure that, before publishing the completed project, each photo record refers to a photograph that has been successfully acquired and is present at the expected location. Records for photographs that could not be acquired MUST be handled appropriately so that the published project does not contain broken file references.

When creating photo records, the Importer MUST:

- Follow the field names, data types and structure specified by the Project JSON Reference.
- Provide all required fields with valid values.
- Use the documented file-reference conventions to identify the corresponding photograph.
- Ensure that the record's metadata describes the correct photograph.
- Avoid publishing records for photographs that were not successfully acquired or saved.

Where information is available from the source system, the Importer SHOULD preserve relevant metadata and populate the corresponding fields defined by the Project JSON Reference. It SHOULD NOT invent values for information that is unknown or unavailable. Where the reference defines how optional or unavailable information is represented, the Importer MUST follow those rules.

The Project JSON Reference determines which fields are required, which are optional and how their values must be represented. The following subsections address file references, identifiers, duplicate records, metadata and acquisition failures in more detail.

4.4. File Names and Duplicate Records

Importers MUST ensure that each photo record refers to the intended photograph and that its file reference resolves to the correct file.

Filename collisions can occur when importing from sources that contain multiple directories with the same filenames. For example, a filesystem Importer may encounter `photo1.jpg` in several different source folders. If these files are collected into a single destination directory without distinguishing their names, one file may overwrite another or a photo record may refer to the wrong photograph.

Where a source can contain such collisions, the Importer SHOULD use a strategy that preserves the distinction between files. This may involve generating unique destination filenames, or using another suitable approach. The chosen approach MUST keep the project data and actual filenames consistent.

Importers SHOULD also avoid creating duplicate photo records for the same photograph unless duplicates are intentional and supported by the project requirements. Where the source contains repeated references to the same photograph, the Importer should determine whether these represent separate intended entries or duplicate data.

Any identifiers, filename constraints or duplicate-handling rules specified by the Project JSON Reference MUST be followed.

5. Completing and Publishing a Project

5.1. The Project Creation Lifecycle

An Importer SHOULD treat project creation as a sequence of distinct stages, from initial preparation through to publication. This makes it easier to identify failures, recover from interruptions and ensure that Photo Manager only discovers completed projects.

A typical lifecycle consists of the following stages:

1. **Prepare:** Determine the destination workspace, choose a suitable project directory and establish a staging location. Allow the user to select the source item/s.
2. **Acquire:** Obtain the photographs and associated information from the source system. The Importer may create photo records before, during or after acquiring the files.
3. **Assemble:** Organise the acquired files and construct the project data in accordance with the Project JSON Reference.
4. **Validate:** Check the project data and confirm that its referenced files and required supporting information are present and accessible.
5. **Publish:** Make the completed project discoverable by Photo Manager using the publication procedure described in this section.
6. **Confirm:** Verify that publication succeeded and report the outcome to the user.

These stages describe the logical lifecycle, not a mandatory internal architecture. An Importer MAY combine stages, perform some operations in a different order where appropriate, or use additional intermediate steps, provided the project remains undiscoverable until it is complete and validated.

If a stage fails, the Importer SHOULD report the problem and retain enough information to support diagnosis or recovery. It MUST NOT publish a project that has failed the required validation checks.

Once publication succeeds, the Importer's responsibility for creating that project is complete. The Importer MUST NOT continue modifying the published project as part of background cleanup or other post-publication activity.

5.2. Keeping Incomplete Projects Undiscoverable

Photo Manager must not discover a project while it is still being assembled or before it has passed validation. An Importer MUST ensure that incomplete projects remain undiscoverable until they are published.

The Importer SHOULD use a staging arrangement that separates projects under construction from published projects. The specific arrangement is an implementation choice, provided it reliably prevents premature discovery.

For example, the Journi Importer uses `importing.json` as the temporary project data filename. Because Photo Manager discovers projects through files named `project.json`, this convention allows the Importer to assemble and validate the project before making it discoverable. Other filenames or mechanisms MAY be used if they provide the same protection.

Importers MUST account for Photo Manager's project discovery behaviour. Merely placing a project in a different directory is not sufficient if the user subsequently changes Photo Manager's workspace and finds a valid `project.json`.

The Importer MUST NOT expose a partially copied or partially assembled project as complete. Where publication involves moving or copying files, it must ensure that the destination cannot be discovered until the handover is complete.

The method used to keep a project undiscoverable SHOULD be documented as part of the Importer's implementation, particularly where it affects recovery from an interrupted import.

5.3. Validating Before Publication

Before publishing a project, the Importer MUST verify that the project is complete and conforms to the applicable Project JSON Reference. Validation should identify problems while the project is still in staging, so they can be corrected without exposing an invalid project to Photo Manager.

Validation SHOULD cover the following areas:

- **Project data:** Confirm that `project.json` can be parsed and complies with the structure, required fields, data types and constraints defined by the Project JSON Reference.
- **File references:** Confirm that each photo record refers to the intended photograph and that the referenced file exists at the expected location.
- **Project contents:** Check that the files required by the project are present and that the project directory structure agrees with the project data.
- **Photo records:** Check that records are consistent with the photographs included in the project and that required information has been provided.
- **Acquisition results:** Ensure that records for photographs that failed to download or save have been handled appropriately and do not leave broken references in the completed project.

Successful JSON parsing alone is not sufficient to establish that a project is valid. Likewise, the presence of all referenced files does not guarantee that the project data complies with the specification.

An Importer MAY implement validation using its own code, suitable libraries, external validation tools or a combination of these approaches. Regardless of the method chosen, it MUST NOT publish a project that fails the required checks.

If validation fails, the Importer SHOULD report the problem in a way that helps the developer or user understand what needs attention. The incomplete project SHOULD remain in staging until it can be corrected, retried or discarded safely.

Validation SHOULD be repeated if changes are made to the project data or files after the initial checks. The project must be in a valid, complete state when it is published.

5.4. Making a Completed Project Discoverable

A project becomes discoverable by Photo Manager when it is published in the configured workspace using the directory structure and project filename expected by Photo Manager.

Before publication, the Importer MUST ensure that the project has passed the required validation checks and that all required files are present. The Importer MUST NOT make an incomplete or invalid project discoverable.

For an Importer using the temporary-file convention described in Section 3.4, publication involves renaming `importing.json` to `project.json` and moving or renaming the project directory into its final location directly beneath the configured workspace.

Other publication methods MAY be used, provided they satisfy Photo Manager's project discovery requirements and prevent the project from becoming discoverable prematurely.

The Importer MUST ensure that the final project directory does not conflict with an existing project. It MUST also handle publication failures without leaving a partially published project that could be mistaken for a completed import.

After publication, the Importer SHOULD confirm that the operation succeeded and report the result to the user. Where practical, it SHOULD verify that the published project can be found and loaded by Photo Manager.

Once the project is discoverable, the Importer MUST NOT continue modifying its project data or files. Any subsequent changes should be made through the appropriate Photo Manager functionality.

5.5. Protecting Existing and Published Projects

An Importer MUST take care not to overwrite, damage or unintentionally modify existing projects when creating or publishing a new project.

Before publication, the Importer SHOULD check whether the intended destination directory already exists. If a project with the same proposed directory name is present, the Importer SHOULD choose a different name or report the conflict for resolution. It MUST NOT overwrite an existing project without explicit user authorisation and a clearly defined procedure for doing so.

A project that has been published and made discoverable by Photo Manager MUST be treated as complete and outside the Importer's control. The Importer MUST NOT modify or replace its `project.json`, photograph files or other project contents as part of a subsequent import.

Importers SHOULD use unique staging directories for separate import operations. This helps prevent concurrent or interrupted imports from interfering with each other and reduces the risk of confusing temporary files with the contents of another project.

If an import is interrupted, the Importer SHOULD recover or clean up its own staging data without affecting existing or published projects. It MUST ensure that recovery procedures do not mistake a published project for an incomplete import.

Where an Importer offers a deliberate replacement or update operation, that operation requires separate safeguards and must not be treated as an ordinary project publication. The operation SHOULD clearly identify the affected project and protect its existing data until the replacement or update can be completed safely.

6. Validation and Testing

6.1. Structural and JSON Validation

Before publishing a project, the Importer MUST ensure that its `project.json` file is valid and that the project satisfies the requirements of the Project JSON Reference.

Photo Manager performs a set of checks when discovering projects. However, these checks do not cover every requirement for a complete, usable project. Importers MUST therefore perform their own validation before publication rather than rely solely on Photo Manager to reject invalid projects.

6.1.1. JSON and Project Structure

The Importer MUST verify that:

- `project.json` can be read and parsed as valid JSON.
- The `workspace` object exists and contains the required fields.
- `workspace.schema` contains the supported schema version.
- `workspace.event` is a non-empty string identifying the project.
- `workspace.image_folder` is a non-empty string identifying an existing photograph directory relative to the project directory.
- The required source information fields — `workspace.importer`, `workspace.source`, `workspace.source_id` and `workspace.source_title` — are present and contain valid, non-empty strings.
- The project data conforms to the applicable Project JSON Reference, including required fields, data types and permitted values.

Completion Status

A published project MUST represent a completed import. The following conditions MUST be satisfied:

- `workspace.registration_status` is `COMPLETE`.
- `workspace.download_status` is `COMPLETE`.
- The `photos` collection contains at least one photo record.
- At least one corresponding photograph file exists in the project and is accessible.

These conditions distinguish a completed, published project from an intermediate project that is still being assembled or downloaded.

6.1.2. Validation Outcome

If any required check fails, the Importer MUST NOT publish the project. It SHOULD report the failure with enough detail to help identify and correct the problem.

Successful JSON parsing alone does not establish that a project is valid. Likewise, passing Photo Manager's discovery checks does not guarantee that every photo record, file reference or metadata value is correct. The Importer remains responsible for validating the completed project against the Project JSON Reference before publication.

6.2. Checking File References

Before publishing a project, the Importer MUST verify that every photo record refers to the correct photograph file and that the file exists at the expected location.

The Importer MUST resolve file references according to the path conventions defined by the Project JSON Reference:

- `workspace.image_folder` identifies the photograph directory relative to the directory containing `project.json`.
- Each photo record's `file` value identifies a photograph relative to that photograph directory.

The Importer SHOULD check every photo record, not just the first record or a sample. For each record, it SHOULD verify that:

- The referenced file exists at the expected location.
- The file can be accessed.
- The filename matches the actual file, including letter case where the filesystem distinguishes uppercase and lowercase names.
- The reference resolves to the intended photograph rather than a different file with the same name.

The Importer MUST NOT publish a project containing broken file references. If a photograph could not be acquired or saved successfully, the Importer SHOULD retry the operation where appropriate or exclude the unavailable photograph and its record from the completed project.

6.2.1. Filename Collisions

Importers MUST take account of filename collisions when collecting photographs from multiple source locations. For example, two source directories may each contain a file named `photo1.jpg`. If both files are copied into the same destination directory without being distinguished, one may overwrite the other.

Where collisions are possible, the Importer MUST generate unique destination filenames or use another suitable method to distinguish the files. The resulting photo records MUST refer to the correct destination files.

6.3. Verifying Metadata and Photo Records

Before publishing a project, the Importer MUST ensure that its photo records accurately describe the photographs included in the project and conform to the Project JSON Reference.

The Importer MUST verify that:

- Each photo record contains the required fields with values of the correct types.
- Required identifiers and other constrained values comply with the Project JSON Reference.
- Each record corresponds to the intended photograph.
- Metadata values are represented in the format required by the specification.
- The project contains at least one photo record, and the records are consistent with the photographs included in the project.

Metadata Accuracy and Preservation

Importers SHOULD preserve relevant information obtained from the source system, including capture dates, locations, descriptions and other contextual metadata supported by the project format.

The Importer MUST NOT invent metadata values or silently substitute unrelated values when information is unavailable. Where the specification defines how missing or optional information must be represented, the Importer MUST follow those rules.

The Importer SHOULD check that metadata has been associated with the correct photograph. For example, a location or description belonging to one photograph must not accidentally be assigned to another because of an incorrect association between source records and downloaded files.

6.3.1. Identifiers and Duplicate Records

Where the Project JSON Reference defines identifiers or uniqueness constraints, the Importer MUST comply with those requirements.

The Importer SHOULD check for unintended duplicate photo records and ensure that filename collisions or repeated source references have not caused records to point to the wrong photographs.

Duplicate entries SHOULD be retained only where they are intentional and consistent with the project requirements.

6.4. Testing

An Importer SHOULD be tested with Photo Manager to confirm that its published projects can be discovered, selected and used successfully. Passing JSON and file-reference checks is necessary, but does not by itself demonstrate that the project works correctly in the application.

Testing SHOULD use a representative project created by the Importer and published through its normal publication procedure.

6.4.1. Project Discovery and Selection

Verify that Photo Manager discovers the published project and displays it in the project-selection interface. Check that the displayed project name is correct and that any available thumbnail refers to the intended photograph.

Confirm that the project is not discoverable while it is still being assembled or before publication.

6.4.2. Project Loading and Photograph Access

Open the published project in Photo Manager and verify that:

- The project loads without errors.
- The expected photographs are available.
- The photographs correspond to their photo records.
- The displayed metadata is associated with the correct photographs.
- Photographs can be navigated and viewed as expected.

Where the project contains a substantial number of photographs, testing SHOULD include enough records to exercise the Importer's normal handling of filenames, metadata and file references.

6.4.3. Metadata Propagation and Initial Values

The project MUST be tested in Photo Manager to verify that metadata propagation functions correctly and that the initial metadata policy values produce sensible results.

Testing SHOULD confirm that:

- Metadata values are propagated to the appropriate photographs according to the project's configured policies.
- The initial propagation settings for each supported metadata field are appropriate for the imported data.
- The resulting metadata is suitable for the user to review and amend within Photo Manager.

If the initial propagation settings produce inappropriate results, the Importer developer SHOULD review the metadata values and policy settings supplied in the project. Testing SHOULD be repeated after changes until the resulting behaviour is satisfactory.

6.4.4. Portability and Repeatability

Where practical, test the project after moving or copying it to another suitable workspace location. This helps identify unintended dependencies on the original source directory, staging location or machine-specific paths.

Importers SHOULD also be tested with representative edge cases, such as source filenames that collide, photographs with incomplete metadata and acquisition failures. These tests help establish that the Importer handles such conditions without publishing an invalid project.

6.4.5. Recording Test Results

Developers SHOULD record significant test failures, their causes and the corrective actions taken. Testing SHOULD be repeated after changes that affect project construction, file handling, metadata mapping or publication.

A project should be considered successfully tested only when it passes the required validation checks and behaves as expected when used in Photo Manager.

7. Development Guidelines and Recommendations

7.1. Language and Technology Independence

Standalone Importers MAY be developed using any programming language, libraries and tools suitable for the task. Photo Manager does not require an Importer to use a particular language, framework or internal architecture.

The essential requirement is that the Importer produces a project that conforms to the applicable Project JSON Reference and meets the workspace, file-management, validation and publication requirements described in this Handbook.

An Importer developer is responsible for choosing an implementation that is reliable and appropriate for the source system, expected collection size and target operating environments.

Implementation choices MUST NOT introduce dependencies into the completed project that prevent Photo Manager from locating or accessing its required files. The project data and file references MUST follow the documented format regardless of the language or libraries used to create them.

Developers SHOULD favour established libraries and straightforward implementation techniques where these reduce complexity and improve reliability. They SHOULD also keep source-specific acquisition logic separate from the code responsible for constructing, validating and publishing the Photo Manager project.

The Handbook describes the expected results and the requirements that must be met. It does not prescribe a particular internal design or require developers to reproduce the implementation of another Importer.

7.2. Cross-Platform Development

Importers intended to run on more than one operating system SHOULD be designed to accommodate differences in filesystem behaviour, path handling and platform configuration.

Developers SHOULD:

- Use the path-handling facilities provided by their chosen language rather than constructing filesystem paths through string concatenation.
- Avoid hard-coded assumptions about the location of the Photo Manager workspace or application configuration files.
- Account for differences in filename case sensitivity and filesystem restrictions when creating destination filenames.
- Handle filesystem errors, including inaccessible directories, insufficient storage and failed file operations.
- Use relative file references in `project.json` as defined by the Project JSON Reference, rather than storing absolute paths tied to a particular computer.
- Test the Importer on each operating system it claims to support.

Filename handling deserves particular attention when an Importer collects files from different source locations. The Importer SHOULD ensure that destination filenames are valid on the intended platforms and that files remain distinguishable without relying on filename differences that a target filesystem may ignore.

An Importer MUST follow the documented workspace and project-format requirements regardless of the operating system on which it runs. Developers SHOULD document any platform limitations that cannot reasonably be avoided.

Developers SHOULD isolate operating-system-specific behaviour in a dedicated module or a small number of clearly defined modules. This may include platform-specific configuration locations, browser discovery, launching external applications and other operations that differ between operating systems.

For example, Photo Manager's `multi_os.py` module centralises several platform-dependent operations. A similar approach in an Importer makes it easier to identify the code that needs to change when porting the application to another operating system.

Platform-specific details SHOULD NOT be scattered throughout the codebase. Common Importer logic SHOULD use the shared functions or interfaces provided by these modules rather than repeatedly implementing operating-system checks.

7.3. Dependencies and Configuration

Importers may rely on third-party libraries, external tools and configuration settings to perform their work. Developers are responsible for managing these dependencies and ensuring that the Importer can be installed, configured and operated reliably.

7.3.1. Dependencies

Developers SHOULD:

- Use established libraries and tools appropriate to the Importer's purpose.
- Keep dependencies to a reasonable minimum to reduce installation complexity and maintenance overhead.
- Document the dependencies required to install and run the Importer.
- Specify dependency versions or compatible version ranges where necessary for reliable operation.
- Ensure that third-party dependencies are obtained from appropriate sources and that their licensing requirements are respected.

An Importer SHOULD report missing or incompatible dependencies clearly rather than failing without an explanation.

7.3.2. Configuration

Configuration values SHOULD be kept separate from the main application logic. This includes settings such as source-system access details, download options, destination choices and other values that may differ between installations.

Developers SHOULD provide a clear and documented way to supply configuration values. Machine-specific paths and environment-specific settings MUST NOT be hard-coded where they should be determined at runtime.

The Importer MUST determine the configured Photo Manager workspace using the documented configuration mechanism described in Section 2. It MUST NOT assume that the workspace is located at a fixed path.

Sensitive configuration values, such as passwords, access tokens and other credentials, SHOULD be handled securely. They SHOULD NOT be embedded in source code or written to diagnostic logs.

7.3.3. Installation and Operation

The Importer's documentation SHOULD explain how to install its dependencies, configure the required settings and start the application. Any required external tools or platform-specific setup SHOULD also be documented.

Developers SHOULD test the documented installation and startup procedure in a clean environment where practical. This helps identify dependencies or configuration that were available on the developer's own machine but have not been documented.

7.4. Separation of Responsibilities

An Importer SHOULD be organised into clearly defined components, with each component responsible for a distinct part of the import process. This makes the code easier to understand, test, maintain and adapt to different source systems.

A typical Importer may separate responsibilities such as:

- **Source access:** connecting to the source system and obtaining information about available photographs.
- **Asset acquisition:** downloading or copying photographs and any required supporting files.
- **Project construction:** creating photo records and assembling the project data in accordance with the Project JSON Reference.
- **Validation:** checking the project data and its file references before publication.
- **Publication:** making the completed project discoverable by Photo Manager.
- **Configuration and diagnostics:** managing settings, reporting progress and recording errors.

These are examples of possible responsibilities, not a required architecture. Developers MAY organise the code differently to suit the language, libraries and complexity of their Importer.

Source-specific acquisition logic SHOULD be kept separate from the code that constructs and publishes the Photo Manager project. This helps prevent changes to one source system from unnecessarily affecting project creation and validation.

Where operating-system-specific behaviour is required, developers SHOULD follow the guidance in Section 7.2 and isolate that behaviour from the main Importer logic.

The Importer MUST remain responsible for producing a project that conforms to the Project JSON Reference and meets the requirements of this Handbook, regardless of how its internal components are organised.

7.5. Logging, Diagnostics and Error Reporting

Importers SHOULD provide useful diagnostic information to help developers and users understand what the Importer is doing and investigate failures. Logging should support troubleshooting without making normal operation unnecessarily complicated.

7.5.1. Logging

Importers SHOULD record significant events during an import, including the start and completion of an import, acquisition failures, validation results and publication outcomes.

Where appropriate, diagnostic messages SHOULD identify the affected project, photograph or processing stage. Developers SHOULD use consistent message formats and distinguish informational messages from warnings and errors.

Logs SHOULD be sufficient to help diagnose problems without recording excessive detail or unnecessarily exposing sensitive information.

7.5.2. Error Handling

Importers SHOULD handle foreseeable errors explicitly, including:

- Failure to access the source system.
- Failed or interrupted photograph acquisition.

- Missing or inaccessible files and directories.
- Insufficient storage space or filesystem errors.
- Invalid project data or failed validation.
- Publication failures or conflicts with existing projects.

When an error occurs, the Importer SHOULD report what failed and, where practical, suggest an appropriate corrective action. It SHOULD distinguish between errors that can be retried and those that require intervention.

An Importer MUST NOT report an import as successfully completed if the project has not been published successfully. It MUST also avoid publishing an incomplete or invalid project following an error.

7.5.3. Diagnostic Information and Recovery

Where an import fails, the Importer SHOULD retain enough diagnostic information to help determine the cause and support recovery. Temporary files and partial downloads SHOULD be handled in a way that avoids confusing them with completed project contents or damaging existing projects.

Sensitive information, including passwords, access tokens and other credentials, MUST NOT be written to logs. Developers SHOULD also consider whether source-system data or personal information needs to be excluded or restricted in diagnostic output.

The Importer's documentation SHOULD explain where diagnostic information can be found and how to report a problem.

7.6. Repeatability and Data Integrity

Importers SHOULD behave predictably when processing the same source data more than once. Repeatable behaviour makes it easier to diagnose problems, recover from interruptions and avoid unintended changes to existing projects.

7.6.1. Repeatable Imports

Where practical, repeating an import SHOULD produce consistent project data and file organisation when the source data and configuration have not changed.

Importers SHOULD avoid creating unintended duplicate files or photo records when an operation is retried. They SHOULD also distinguish between a new import and an attempt to resume or recover an interrupted import.

The method used to handle repeated or interrupted imports is an implementation choice. Regardless of the method chosen, the Importer MUST avoid overwriting or modifying existing published projects unintentionally.

7.6.2. Protecting Data Integrity

Importers MUST ensure that the project data accurately represents the photographs and associated information included in the completed project.

Developers SHOULD take precautions against partial writes, interrupted downloads, filename collisions and other failures that could leave the project inconsistent. Where a file operation fails, the Importer SHOULD detect the failure and avoid treating the affected file or project as complete.

The Importer MUST validate the completed project before publication and MUST NOT publish a project that fails the required checks.

7.6.3. Recovery and Cleanup

Importers SHOULD handle interrupted operations in a way that permits safe recovery or cleanup without damaging existing projects.

Temporary files and incomplete project data SHOULD remain distinguishable from published project contents. Cleanup procedures MUST be limited to files and directories that the Importer can safely identify as belonging to its own unfinished operation.

Where recovery is supported, the Importer SHOULD verify the state of existing files and project data before resuming work rather than assuming that an interrupted operation completed successfully.

7.7. Documentation and Maintainability

Importers SHOULD be documented so that another developer or user can understand their purpose, install and configure them, operate them and diagnose common problems.

7.7.1. Documentation

Documentation SHOULD explain:

- The Importer's purpose and the source system it supports.
- The prerequisites and dependencies required for installation.
- How to configure and run the Importer.
- How the Photo Manager workspace is located and how projects are published.
- Where diagnostic information can be found and how common failures can be investigated.
- Any known limitations or platform-specific requirements.

Developers SHOULD keep documentation consistent with the actual implementation and update it when behaviour, configuration or dependencies change.

7.7.2. Maintainability

Importers SHOULD use clear naming, consistent coding conventions and a structure appropriate to their complexity. Important design decisions and non-obvious behaviour SHOULD be documented where this helps future maintenance.

Developers SHOULD avoid unnecessary complexity and duplication. Shared behaviour SHOULD be implemented consistently, and operating-system-specific code SHOULD be isolated as described in Section 7.2.

Importers SHOULD be tested after changes that affect project construction, validation, file handling, metadata or publication. This helps ensure that maintenance does not introduce regressions that result in invalid projects or unintended changes to existing data.

7.7.3. Licensing

Importers SHOULD include a clear licence specifying the terms under which their code may be used, modified and redistributed. The choice of licence is left to the Importer developer.

Developers MUST ensure that the licences of third-party libraries, tools and other included components are respected. Where an Importer is distributed to others, its documentation SHOULD identify the applicable licence and any relevant third-party licensing requirements.

The choice of licence does not alter the requirement for the Importer to produce projects that conform to the applicable Project JSON Reference.

8. Practical Examples

8.1. Staging the Import and Creating importing.json

Before photographs are processed, the Importer needs a working area in which to assemble the project. This area holds the staged photographs and a working JSON file that records the progress of the import.

The Filesystem Importer uses the following directory structure beneath the configured Photo Manager workspace:

```
PhotoManagerWorkspace/
└── Importers/
    └── Filesystem/
        ├── importing.json
        └── photos/
```

The `Importers/Filesystem` directory is the staging area. It is separate from the directories used for completed Photo Manager projects.

The `photos` directory is created when the initial .json checkpoint is written. It is initially empty. Photographs are added to it as they are processed by the import loop.

8.1.1. Creating the Initial Checkpoint

Once the source directory has been selected, directory levels classified and the Event Name confirmed, the Importer creates the initial checkpoint.

In Filesystem Importer this is handled by `checkpoint_writer.py`, using the `write_initial_checkpoint()` function.

The function creates the staging directory and its `photos` subdirectory, then writes the initial `importing.json` file.

`Importing.json` contains three parts of the Photo Manager project structure:

- `workspace` — identifies the project, its source and its current status.
- `metadata_policies` — defines the initial metadata propagation settings.
- `photos` — initially an empty list, which will be populated as photographs are processed.

It also contains a `filesystem_importer` object holding information needed by Filesystem Importer to manage the import and recover it if interrupted. This additional object is temporary importer-specific data and is not included in the final published project.

8.1.2. Example importing.json

The following example shows the initial checkpoint structure. The paths, Event Name, timestamp and source scan details are illustrative. In a working Importer, these values are generated from the selected source directory and the actual scan results.

```
{
    "workspace": {
        "schema": 2,
        "importer": "filesystem-importer/0.1.0",
        "source": "Filesystem",
        "source_id": "/home/user/Pictures/Family-Holiday",
        "source_title": "Family-Holiday",
        "registration_status": "NOT_STARTED",
        "download_status": "NOT_STARTED",
        "event": "Family Holiday",
        "image_folder": "photos",
        "created_at": "2026-11-01T03:30:00.000Z"
    },
    "metadata_policies": {
        "event": {
            "propagate": "none",
            "priority": [],
            "default": ""
        },
        "year": {
            "propagate": "none",
            "priority": [],
            "default": ""
        },
        "location": {
            "propagate": "location",
            "priority": [],
            "default": ""
        },
        "info": {
            "propagate": "group",
            "priority": [],
            "default": ""
        }
    },
    "photos": [],
    "filesystem_importer": {
        "classification_limit": 10,
        "directory_classifications": {
            "1": "Event",
            "2": "Location",
            "3": "Year"
        },
```

```
        "source_manifest": {
            "algorithm": "sha256",
            "fingerprint": "<generated SHA-256 fingerprint>",
            "candidate_count": 33
        }
    }
}
```

The fingerprint is a hash value for the selected directory structure and is used to confirm that the selected directory structure has not been modified if an import is interrupted and then resumed.

8.1.3. The Importer-Specific Data

The `filesystem_importer` object is used by Filesystem Importer to retain information that is needed during processing.

In this example:

- `classification_limit` records the maximum number of directory levels supported by the classification interface.
- `directory_classifications` records the category assigned to each directory level. These values are needed when resolving metadata from the source directory structure.
- `source_manifest` records the details of the original source scan. The candidate count provides the total number of photographs to process, while the fingerprint allows the Importer to check that the source has not changed when recovering an interrupted import.

This information is specific to the Filesystem Importer. Another Importer may require different internal information, or may not need an additional object at all.

8.1.4. Why Use importing.json?

The working file is deliberately named `importing.json`, rather than `project.json`.

Photo Manager looks for `project.json` when discovering available projects. Using a different filename allows the Importer to keep its working data in the workspace without exposing an incomplete project to Photo Manager.

The `importing.json` file also acts as a recovery checkpoint. As photographs are processed, the Importer updates the checkpoint so that work already completed can be identified if the operation is interrupted.

The initial checkpoint therefore contains an empty `photos` array. Individual photo records are added as the import loop processes the candidates. The checkpoint is saved as processing progresses.

The working file will eventually be used to prepare the completed project. That preparation and publication process is covered later in this section.

8.2. Acquiring and Validating Files and Constructing Photo Records

This section follows the processing of an individual photograph through the Filesystem Importer. It shows how a source file is copied into the staging area, tested, and represented by a photo record containing the information available to the Importer.

The precise acquisition and validation process will depend on the source system and the requirements of each Importer. The Filesystem Importer provides a practical example of one approach. In particular, its use of directory classifications to propose metadata is specific to importing photographs from an existing directory structure. Other Importers may obtain equivalent information from a source service, database, download manifest or other source-specific information.

The general objective is the same: acquire the photograph reliably, construct an accurate photo record, and preserve useful information that can help the user understand and organise the photograph.

8.2.1. Processing one photograph

The Filesystem Importer processes photographs individually in its importing loop. For each candidate, it:

1. Copies the source photograph into the staging directory.
2. Tests whether the staged photograph supports the EXIF operation required by Photo Manager.
3. Gathers the available metadata and source information.
4. Proposes values for the Photo Manager metadata fields.
5. Completes the photo record and appends it to the `photos` list in `importing.json`.
6. Saves the updated recovery checkpoint.

This process is repeated for each candidate photograph. The checkpoint is updated as the import progresses, allowing processing to resume after an interruption without having to repeat candidates already recorded.

8.2.2. Copying and testing the photograph

The Filesystem Importer copies each candidate into the staging `photos` directory. It preserves the original filename where possible. If another photograph in the staging directory already has that filename, it adds a numbered suffix to create a unique destination filename.

The original source photograph is not modified. The staged copy is used for subsequent processing.

The Importer then tests whether it can write an EXIF `UserComment` value to the staged photograph and restore the original comment successfully. This is important because Photo Manager must subsequently write metadata into the photograph's EXIF data.

Where an original UserComment exists, the test verifies that it is restored after the temporary test value has been written. A successful test therefore establishes that the staged copy supports the required operation without losing its existing comment.

This is a validation step specific to the Filesystem Importer's requirements. Other Importers may need different checks, depending on the formats they acquire. In all cases Photo Manager will write the exifdata UserComment to each photo file and there is no value in providing files that do not support UserComment as part of a Photo Manager project.

If the copy or test fails, the staged copy is removed. An incomplete or unusable file must not be left in the staging directory and treated as a successfully acquired photograph.

8.2.3. Gathering information for the photo record

After a photograph has been staged successfully, the Filesystem Importer gathers the information needed to construct its photo record.

It uses several sources:

- The original source path, which identifies the photograph in its source collection.
- The staged filename, which identifies the copied photograph within the project.
- EXIF information, including `UserComment`, `DateTimeOriginal` and GPS coordinates.
- The filesystem timestamp of the source photograph.
- The photograph's position in the original directory structure.
- The Event Name and directory classifications supplied during the import setup.

The source photograph is used to obtain its filesystem timestamp and directory position. EXIF information is read from the staged copy.

These inputs are gathered before the proposed metadata values are resolved. Keeping the source information separate from the resulting proposals makes it possible to apply different rules to different metadata fields.

An Importer working with an online service, for example, might gather descriptions, dates, location details and identifiers from the service's records rather than from EXIF data or directory names.

8.2.4. Proposing metadata values

The photo record contains four user-editable metadata fields: `event`, `year`, `location` and `info`.

An Importer should make sensible use of the information available to it when populating these fields. However, automatically extracted or inferred values should be treated as proposals, not as guaranteed facts.

There are two important considerations.

First, information should not be discarded merely because the Importer cannot confidently interpret it. A photograph may contain a useful comment or description even when its format does not match the Importer's expected structure.

Second, the Importer should avoid presenting guesses as established information. Where a value cannot be determined reliably, it is preferable to leave the field empty or preserve the original information for the user to review.

Where the Importer's workflow permits, uncertain or alternative information should be made available to the user rather than silently discarded. The user is better placed to recognise the significance of a name, description or event reference that the Importer cannot interpret reliably.

The Filesystem Importer applies these principles to the four metadata fields using the following approach.

Info

The Importer first examines the EXIF `UserComment`.

If the comment can be confidently interpreted in the established Photo Manager format, its Info component is extracted. If the structure cannot be recognised or the extracted fields do not pass validation, the entire original comment is retained as the proposed Info value.

This is an important example of preserving information rather than discarding it because its structure is unexpected.

If there is no viable UserComment, the Filesystem Importer looks for a directory classified as Info. As a final source, it considers the filename, excluding recognised camera-generated names and other uninformative patterns.

Filename evaluation is used only to determine whether a name is suitable as a proposed description. The original filename remains unchanged.

Year

The Filesystem Importer first looks for a Year component in UserComment. If one can be extracted and validated, it is preserved.

Otherwise, the Importer uses the calendar year from EXIF `DateTimeOriginal`. If that is unavailable or invalid, it uses the year from the source file's filesystem timestamp.

Only when those sources do not provide a usable value does the Importer look for an applicable directory classified as Year.

This directory-based fallback is specific to the Filesystem Importer. An Importer using a service that supplies event dates or other temporal information would use the corresponding source data instead.

Event

The Filesystem Importer first looks for an Event component in UserComment. If none is available, it uses the nearest applicable directory classified as Event.

If no such directory provides a value, it uses the Event Name confirmed by the user when setting up the import.

The filename is not used as a source for Event because it is far more likely to describe an individual photograph rather than the occasion or activity.

Location

The Importer first looks for a Location in UserComment and if present, that name is validated as an actual location through geocoding..

If no usable value is available, it examines the photograph's GPS coordinates. Valid coordinates are converted into a human-readable location name through reverse geocoding.

If coordinates are unavailable, or geocoding does not produce a usable result, the Filesystem Importer examines directories classified as Location. It checks the nearest applicable directory first and can continue to parent directories where geocoding does not validate a directory name as a location.

The Importer also retains the original latitude and longitude in the photo record. These are used to calculate `location_key`, which provides a geographic grouping value for Photo Manager.

Where coordinates are unavailable, the Filesystem Importer uses the applicable Location directory name as the grouping key. This is another example of directory inheritance specific to this Importer.

8.2.5. Understanding the grouping fields

The photo record contains several fields that are related to location and grouping, but they serve different purposes:

- `lat` and `lng` contain the original GPS coordinates, where available.
- `geo_location` contains the human-readable location resolved from GPS coordinates.
- `location` contains the proposed, user-editable Location value.
- `location_key` identifies a geographic grouping used by Photo Manager.
- `moment_id` identifies the group to which the photograph belongs for metadata propagation.

For the Filesystem Importer, `moment_id` is set to the name of the photograph's containing directory.

When valid GPS coordinates are available, `location_key` is calculated from the coordinates rounded to two decimal places. Otherwise, it is inherited from the nearest directory classified as Location and validated as an actual location through geocoding. If neither source is available, the key remains unset.

These values should not be confused. For example, a photograph may have GPS coordinates and a coordinate-based `location_key`, while its user-editable `location` field contains a place name extracted from UserComment.

Other Importers may use different methods to determine grouping identifiers, but they must populate the corresponding Photo Manager fields consistently with the Project JSON Reference and the intended propagation behaviour.

8.2.6. Constructing the photo record

Once the information has been gathered and the metadata proposals determined, the Filesystem Importer completes the photo record.

A successful record contains the source identifier, staged filename, processing state and proposed metadata values. The following example illustrates the resulting structure. The values are illustrative.

```
{
    "guid": "/source/Family-Holiday/Australia/Day 1/photo.jpg",
    "moment_id": "Day 1",
    "lat": null,
    "lng": null,
    "file": "photo.jpg",
    "downloaded": true,
    "retry_count": 0,
    "status": "pending",
    "exif_written": false,
    "last_updated": "2026-10-31T08:05:39.079Z",
    "location_key": "Australia",
    "geo_location": null,
    "event": "Family Holiday",
    "year": "2026",
    "location": "Brisbane",
    "info": "Leaving for the first day's ride"
}
```

Some fields describe the photograph and its source:

- `guid` identifies the original source photograph. The Filesystem Importer uses its resolved source path.
- `file` identifies the staged copy, relative to the directory specified by `workspace.image_folder`.
- `moment_id` identifies the containing folfder.

Other fields describe the processing state:

- `downloaded` is `true` because the photograph was successfully staged.
- `retry_count` is initially `0`.
- `status` is initially `pending`.
- `exif_written` is `false` because Photo Manager has not written the proposed metadata back into the photograph.
- `last_updated` records when the photo record was created or updated, using a UTC timestamp.

The remaining fields contain the proposed metadata and grouping information. Empty strings and `null` values are used where information is unavailable, in accordance with the Project JSON Reference.

8.2.7. Handling a candidate that fails testing

A candidate that fails the staging or EXIF capability test is still recorded in the recovery checkpoint. This allows the Importer to account for every candidate and report unsuccessful processing.

The Filesystem Importer constructs a record with the source path as its `guid`, `downloaded` set to `false`, and an empty `file` value. Metadata fields remain empty because the candidate did not proceed through the metadata-processing steps.

For example:

```
{
    "guid": "/source/Family-Holiday/Australia/Day 1/problem-photo.jpg",
    "moment_id": "",
    "lat": null,
    "lng": null,
    "file": "",
    "downloaded": false,
    "retry_count": 0,
    "status": "pending",
    "exif_written": false,
    "last_updated": "2026-10-31T08:05:39.615Z",
    "location_key": null,
    "geo_location": null,
    "event": "",
    "year": "",
    "location": "",
    "info": ""
}
```

The record is appended to `importing.json` and saved with the rest of the recovery checkpoint. It is not intended to be included in the completed project. The project-preparation process described later removes records for photographs that were not successfully acquired.

8.2.8. Appending the record and saving progress

Whether the candidate succeeds or fails, its record is appended to the `photos` list in `importing.json`.

The Filesystem Importer then saves the updated recovery checkpoint. It uses an atomic save operation: the new JSON data is written to a temporary file and then replaces the previous checkpoint. This reduces the risk of leaving a partially written checkpoint if the application is interrupted during the save.

The importing loop updates the progress display at the beginning of each iteration. After the record has been appended and the checkpoint saved, processing returns to the start of the loop for the next candidate.

This completes the processing of one candidate photograph. The same procedure is repeated until all candidates have been accounted for.

At this stage, `importing.json` contains the working photo records and the results of the acquisition process. It is still a recovery checkpoint, not the completed project. Preparing and validating the final project data is covered in the following section.

8.3. Preparing and Validating the Completed Project

When the importing loop has processed all candidate photographs, the Filesystem Importer has a working checkpoint containing the results of the import. This file is named `importing.json` and includes both successfully acquired photographs and records for candidates that failed processing.

The checkpoint also contains information used internally by the Filesystem Importer to support recovery and verify the import. This information is not part of the published Photo Manager project.

The next stage prepares the project data for publication and checks that the resulting project is complete and suitable for use by Photo Manager.

8.3.1. Preparing the project data

The Filesystem Importer uses the `project_preparation.py` module to create a new file named `prepared.json` from `importing.json`.

This is a separate file rather than an in-place modification of the recovery checkpoint. Keeping the original checkpoint until publication succeeds provides a way to retain the import results if project preparation or finalisation encounters a problem.

The preparation process makes several changes.

Remove unsuccessful photo records

Only photographs that were successfully staged are included in the completed project.

The Filesystem Importer identifies these records through their `downloaded` field:

- Records with `downloaded` set to `true` are retained.
- Records with `downloaded` set to `false` are excluded.

The failed records have served their purpose in the recovery checkpoint by recording the outcome of each candidate. They must not be carried into the published project because they do not identify usable staged photographs.

Remove Filesystem Importer-specific data

The top-level `filesystem_importer` object is used internally by the Filesystem Importer. It contains information such as the directory classifications and source manifest.

This object is removed from the prepared project data. It is not part of the Photo Manager project structure defined by the Project JSON Reference.

An Importer may maintain its own internal data while creating a project, but it must not introduce undocumented fields into the published project.

Update project completion status

The Filesystem Importer changes the workspace processing status fields to indicate that the project has been completed:

```
"registration_status": "COMPLETE",
"download_status": "COMPLETE"
```

These values describe the completed state of the import. They are not a substitute for checking that the project data and files are valid.

The remaining project information, metadata policies and successful photo records are retained in the prepared data.

8.3.2. Resulting project structure

The prepared file has the same main structure expected by Photo Manager:


```
{
    "workspace": {
        "schema": 2,
        "importer": "filesystem-importer/0.1.0",
        "source": "Filesystem",
        "source_id": "/source/Family-Holiday",
        "source_title": "Family-Holiday",
        "registration_status": "COMPLETE",
        "download_status": "COMPLETE",
        "event": "Family Holiday",
        "image_folder": "photos",
        "created_at": "2026-10-31T08:05:39.060Z"
    },
    "metadata_policies": {
        "event": {
            "propagate": "none",
            "priority": [],
            "default": ""
        },
        "year": {
            "propagate": "none",
            "priority": [],
            "default": ""
        },
        "location": {
            "propagate": "location",
            "priority": [],
            "default": ""
        },
        "info": {
            "propagate": "group",
            "priority": [],
            "default": ""
        }
    },
    "photos": [
        {
            "guid": "/source/Family-Holiday/Australia/photo.jpg",
            "moment_id": "Day 1",
            "lat": null,
            "lng": null,
            "file": "photo.jpg",
            "downloaded": true,
            "retry_count": 0,
            "status": "pending",
            "exif_written": false,
            "last_updated": "2026-10-31T08:05:39.079Z",
            "location_key": "Australia",
            "geo_location": null,
            "event": "Family Holiday",
            "year": "2026",
            "location": "Brisbane",
```

```
            "info": "Leaving for the first day's ride"
        }
    ]
}
```

This is an abbreviated illustrative example containing one photo record. The actual file must contain all required fields and records for the photographs included in the project.

Notice that `workspace.image_folder` identifies the `photos` directory, while the photo record's `file` field contains the filename relative to that directory. The original source path is retained in `guid` as the source identifier, but it is not used as the project's photograph file reference.

8.3.3. Validating the prepared project

Preparing the JSON file is not sufficient by itself. Before publication, the Importer must confirm that the project data and the files it references are consistent.

The Filesystem Importer performs its finalisation checks through `project_finalization.py`. These checks include the prepared photo records and the staged files they reference.

The validation should establish that:

- The prepared JSON can be read and parsed.
- The project contains the required top-level objects: `workspace`, `metadata_policies` and `photos`.
- The workspace and metadata policy values conform to the Project JSON Reference.
- Each retained photo record contains the required fields and appropriate value types.
- No records remain for photographs that failed to be acquired.
- Each photo record's `file` value identifies a staged photograph that actually exists in the project's `photos` directory.
- The project data and the staged files agree, so that each record refers to the intended photograph.

These checks are not unique to the Filesystem Importer. Every Importer must ensure that its completed project conforms to the applicable Project JSON Reference and that its photo records refer to the correct, accessible files. The particular code and sequence used to perform the checks will depend on the Importer's implementation.

The Project JSON Reference remains the authority for required fields, permitted values and data types. Developers should validate against that specification rather than assume that a file is valid simply because it is well-formed JSON or resembles an existing project.

If the prepared project fails validation, it must not be published. The problem should be reported and resolved before the Importer proceeds.

8.3.4. Keeping the working and prepared files separate

At the end of this stage, the Filesystem Importer has two distinct files:

- `importing.json` — the original working checkpoint, including the importer's internal data and records for failed candidates.
- `prepared.json` — the cleaned project data containing only successfully acquired photographs and the completed workspace status values.

Neither file is yet named `project.json`, so the prepared project has not been published for discovery by Photo Manager.

The original checkpoint is retained until publication has succeeded. The next stage moves the completed project into its final workspace location and makes it discoverable by Photo Manager.

8.4. Publishing the Completed Project

Once the prepared project has passed validation, it can be published to the Photo Manager workspace.

Publication is the point at which the project becomes available for Photo Manager to discover. Until this happens, the project remains under the control of the Importer and must not be presented as a completed project.

The Filesystem Importer uses a staging directory beneath the workspace's `Importers` directory. Its finalisation process moves the completed project into its own directory directly beneath the configured Photo Manager workspace.

Other Importers may use different staging arrangements or publication methods. Whatever method is chosen, the essential requirement is that Photo Manager must not discover the project until all required files and project data are in place and validation has succeeded.

8.4.1. Choosing the final project directory

Every published project must reside in its own directory directly beneath the configured Photo Manager workspace.

For example, the completed project might be organised as follows:

```
PhotoManagerWorkspace/
├── Family Holiday/
│   ├── project.json
│   └── photos/
│       ├── photo1.jpg
│       └── photo2.jpg
│
└── Importers/
    └── Filesystem/
        └── ...
```

The final project directory name identifies its location in the filesystem. It does not have to match the Event Name stored in `workspace.event`.

The Filesystem Importer chooses an available directory name for the completed project. It must avoid conflicts with existing directories so that an import cannot overwrite or damage a previously published project.

All Importers should take the same precaution. An existing project must not be replaced or modified unintentionally during publication.

8.4.2. Finalising the Filesystem Importer project

The Filesystem Importer performs finalisation through its `project_finalization.py` module.

The process is as follows:

1. The finalisation code checks the prepared project and the staged files it references. This provides a final safeguard before publication.
2. The completed project directory is moved into a unique directory directly beneath the configured Photo Manager workspace.
3. The prepared project data file is renamed from `prepared.json` to `project.json`.
4. Once publication has succeeded, the original `importing.json` recovery checkpoint is removed.

The rename to `project.json` is significant. Photo Manager discovers projects through this filename. Until the rename occurs, the project remains undiscoverable as a completed project.

The final directory now contains the project data file and the photograph directory identified by `workspace.image_folder`.

8.4.3. Confirming publication

The Importer should not report a successful import merely because the finalisation operation was attempted. It should confirm that the project was published successfully.

The Filesystem Importer reports the completion result to the user, including the import statistics. The user can then acknowledge the result and return to the initial application screen.

During development, the published project should also be tested with Photo Manager. Confirm that it is discovered, can be opened, and that the photographs and their associated metadata are available as expected.

This application-level check should form part of the testing of any new Importer.

8.4.4. After publication

Once the project has been published and made discoverable, responsibility for its contents passes to Photo Manager.

The Importer must not continue modifying the published project, its `project.json` file or its photographs. Any subsequent changes should be made through the appropriate Photo Manager functionality.

The Importer may perform its own cleanup of temporary data that is no longer required, provided that this does not alter the published project.

This completes the Filesystem Importer's import process. It has acquired and tested the photographs, constructed the photo records, prepared and validated the project data, and published the completed project in the configured workspace.

The result is a normal Photo Manager project, independent of the Importer that created it.

8.5. Lessons from the Filesystem Importer

The Filesystem Importer provides a working example of how an Importer can acquire photographs, construct Photo Manager records, and publish a completed project. Its implementation also illustrates several practical lessons that are useful when developing other Importers.

These are lessons drawn from this particular implementation, not a prescribed architecture. An Importer dealing with a different source, service, or acquisition method may need a different approach.

8.5.1. Keep acquisition separate from publication

The Filesystem Importer does not make a project available to Photo Manager as soon as photographs start arriving.

Instead, it maintains a working checkpoint while acquisition is in progress. Once acquisition is complete, it prepares and validates a clean project before publishing it.

This separation has two benefits:

- An incomplete import is not mistaken for a completed project.
- Photo Manager only discovers the project once it has reached the publication stage.

The exact filenames and directory arrangement are implementation choices. The important point is to establish a clear boundary between work in progress and a published project.

8.5.2. Preserve enough information to recover

An import can be interrupted by an application failure, a system shutdown, or a problem with an individual photograph.

The Filesystem Importer addresses this by saving its checkpoint as the import progresses. Each candidate is accounted for, including candidates that could not be acquired successfully.

This makes it possible to distinguish work that has succeeded from work that has failed, rather than relying on the application remaining open until the entire import is finished.

A useful general principle is to decide early what information will be needed to resume or diagnose an interrupted import, and to preserve it as the work proceeds.

8.5.3. Do not modify the original source unnecessarily

The Filesystem Importer copies photographs into its staging area before testing or processing them. Its EXIF capability test is performed on the staged copy, and the original source photograph is left unchanged.

This is a useful precaution for any Importer that handles files supplied by the user. Work on a copy where practical, and avoid making changes to the original material unless modification is an explicit and necessary part of the import process.

8.5.4. Treat metadata as proposals, not unquestionable facts

Information associated with a photograph is not always complete or consistently formatted.

The Filesystem Importer attempts to extract useful values from available comments and EXIF data, and uses directory information where appropriate. However, it does not assume that every value can be interpreted with certainty.

Where a comment cannot be confidently interpreted, the original comment is retained as Info rather than silently discarded. This leaves potentially useful information available for the user to review.

The broader lesson is to distinguish between information that has been extracted confidently, information that has been inferred, and information that could not be interpreted. An Importer should not turn an uncertain interpretation into an apparently authoritative value.

The Filesystem Importer's directory classification and inheritance methods are specific to importing from a filesystem. Another Importer may obtain context from a completely different source.

8.5.5. Validate acquired files before recording them as successful

A file being present in a staging directory does not, by itself, establish that it has been acquired successfully.

The Filesystem Importer tests the staged photograph's ability to retain the EXIF UserComment field before completing its photo record. A candidate that fails this test is recorded as unsuccessful and is not included in the published project.

The general lesson is to define what successful acquisition means for the particular Importer and test for it before treating an item as successfully acquired.

8.5.6. Make progress and failure visible

The Filesystem Importer updates its progress display at the start of each import-loop iteration. It also retains failed candidates in the working checkpoint.

This gives the user a view of progress while preserving a record of what happened during acquisition.

Progress reporting should reflect the actual state of the work. It should not imply that an item has completed successfully before the relevant processing and checkpoint operations have finished.

8.5.7. Keep Importer-specific information out of the published project

The working checkpoint contains information needed internally by the Filesystem Importer, including its directory classifications and source manifest.

That information is removed when the clean project is prepared for publication. The published project contains the standard Photo Manager structures and records, rather than internal details of the Importer's implementation.

This separation helps keep the project format independent of the Importer that created it. Photo Manager should be able to work with a valid project without needing to understand how the Importer acquired or interpreted its contents.

8.5.8. Publish only after the project is ready

The Filesystem Importer validates the prepared project and its referenced files before publication. It then places the project in the Photo Manager workspace and gives it the `project.json` filename that Photo Manager uses to discover projects.

This makes publication an explicit final step rather than an incidental consequence of creating a JSON file.

The general principle is to ensure that the published project is complete, internally consistent, and ready for Photo Manager to use. Once published, the Importer should not continue modifying it.

8.5.9. Final observation

The Filesystem Importer demonstrates one practical way to implement the Importer workflow. Its filesystem-specific features, including directory classification and metadata inheritance, are not requirements for other Importers.

The requirements that matter are the ones established by the Photo Manager project format and the Importer interface. The implementation details should be chosen to suit the source being imported, while ensuring that the result is a valid, complete, and usable Photo Manager project.

9. Troubleshooting Photo Manager Compatibility

This section covers a small number of problems that may arise when a project produced by an Importer is handed over to Photo Manager.

It is not intended to describe general software troubleshooting techniques. It concentrates on the points where the Importer and Photo Manager interact.

9.1. Project is not discovered by Photo Manager

Check the following:

- The Importer has published the project into the workspace configured in Photo Manager's settings.
- The project directory is located directly beneath the configured workspace.
- The project directory contains a file named exactly `project.json`.

A project that contains a valid `project.json` file will not be discovered if it is not published in or under the workspace configured in Photo Manager's settings

9.2. Project is discovered but cannot be opened

Check the published `project.json` file.

- Confirm that it can be parsed as valid JSON.
- Confirm that it contains the three required top-level members: `workspace`, `metadata_policies` and `photos`.
- Confirm that all required fields are present and use the correct data types.
- Check that field names and permitted values conform to the applicable Project JSON Reference.
- Confirm that the `workspace.schema` value identifies a schema version supported by the installed Photo Manager version.

Successful JSON parsing alone does not establish that a project is valid. The structure and contents must also conform to the specification.

9.3. Project opens but photographs are missing

Check the relationship between the project directory and its photograph files.

Photo Manager resolves photograph paths using two fields:

- `workspace.image_folder` identifies the photograph directory relative to the directory containing `project.json`.
- Each photo record's `file` value identifies the photograph filename relative to that photograph directory.

For every photograph record, check that:

- The referenced file actually exists.
- The filename matches the actual filename, including its extension and letter case.
- The file can be accessed.
- The record has `downloaded` set to `true`.

Also check that records for photographs which failed acquisition have not been included in the published project.

9.4. Project opens but metadata is missing or unexpected

Inspect the metadata values in the published `project.json`.

Check that:

- The individual photo records contain the intended values for Event, Year, Location and Info.
- The `metadata_policies` entries contain the intended propagation settings.

- The Importer has placed values in the correct fields rather than relying on information embedded in the photograph to supply missing project data.

This helps distinguish a problem with the metadata produced by the Importer from an expectation about how Photo Manager should display or propagate that metadata.

9.5. Final check

When investigating an integration problem, examine the actual published project directory and its `project.json` file rather than relying solely on the Importer's internal records or completion message.

The **Project JSON Reference** is the authoritative source for the project structure and field definitions. The Handbook describes the expected directory layout and how the project is handed over to Photo Manager.

Once the published project conforms to those requirements, testing it in Photo Manager provides the final confirmation that the integration is working.

10. Reference

10.1. Relevant Specifications

The following documents provide the principal technical references for Importer development.

10.1.1. Photo Manager Project JSON Reference

Document Version: 1.0
Project JSON Schema Version: 2
Status: Published

This is the authoritative specification for the structure and contents of `project.json`.

It defines the required fields, data types, permitted values and constraints for Photo Manager projects.

Every Importer MUST produce project data that conforms to the applicable version of this specification.

- Read or download the PDF
- Download the editable ODT document

Photo Manager Core Design Specification

Document Version: 1.3
Status: Draft

This document describes the internal design and architecture of the Photo Manager Application. It provides useful context for Importer developers, particularly regarding project discovery, loading, validation and workspace management.

The Core Design Specification does not define the structure of `project.json`. That remains the responsibility of the Project JSON Reference.

As this document is marked Draft, developers should check that they are consulting the applicable version for their target Photo Manager release.

- Read the Core Design Specification

Source System Documentation

An Importer may need to consult additional documentation relating to the source from which it obtains photographs and associated information.

This may include source-system API documentation, authentication requirements, data formats, asset identifiers, metadata definitions and usage restrictions.

These documents are specific to the source system and are selected by the Importer developer. They do not alter the requirements of the Photo Manager project format.

10.1.2. Using the Specifications

The Project JSON Reference defines what an Importer must produce. The Photo Manager Core Design Specification helps explain how the application uses the resulting project. Source-system documentation explains how the Importer obtains its information and assets.

This Handbook provides practical guidance on applying those requirements. It does not replace or override any of the specifications.

10.2. Glossary

Asset

A photograph obtained from the source system and represented in the Photo Manager project by a photo record.

Importer

A standalone application that obtains photographs and associated information from a source system and prepares them as a Photo Manager project.

Metadata Policies

The project-wide settings that define how Event, Year, Location and Info metadata are propagated when a user edits a photograph.

Each policy contains a `propagate` value. Optional `priority` and `default` values may also be used by the Importer when determining initial metadata values. Photo Manager does not use these optional fields.

Photo Manager Workspace

The configured directory in which Photo Manager stores and discovers completed projects. Each completed project resides in its own directory directly beneath the workspace.

Photo Record

An individual object in the `photos` array of `project.json`. It describes one asset and contains its identifiers, file reference, download and processing status, geographic information and editable metadata.

Project

A complete collection of project data, photographs and any required supporting files, organised so that Photo Manager can discover and use it.

Project Directory

The directory containing an individual project. It contains `project.json`, the photograph directory and any other files required by the project.

Project Discovery

The process by which Photo Manager identifies available project directories and determines whether their project data is sufficiently valid for presentation as usable projects.

Project JSON Reference

The authoritative specification defining the structure, fields, data types, permitted values and constraints of `project.json`.

Project Validation

The process of checking that project data conforms to the Project JSON Reference and that the photographs and other required files referenced by the project are present and accessible.

An Importer performs its own validation before publication. Photo Manager also validates projects when discovering and loading them.

project.json

The project data file used by Photo Manager to identify a project and locate its photographs. It contains the `workspace`, `metadata_policies` and `photos` top-level members.

Published Project

A completed and validated project placed in the configured Photo Manager workspace using the required directory structure and filename. Once published, it becomes available for Photo Manager to discover and load.

Relative Path

A filesystem path expressed in relation to another directory rather than as an absolute path tied to a particular computer. Photo Manager uses relative paths to locate project photographs, allowing a project directory to be moved without rewriting its internal file references.

Schema Version

The version of the project data structure identified by the `workspace.schema` field. It allows Photo Manager to determine whether the project format is compatible with the application.

Source Asset Identifier (`guid`)

The identifier used to identify an asset within its originating source. Its format and meaning are defined by the Importer. Photo Manager treats it as an identifier and does not interpret its internal format.

Staging Area

A temporary location used by an Importer while acquiring photographs, constructing project data and performing validation. A project in staging must not be discoverable by Photo Manager as a completed project.

Group Identifier (`moment_id`)

The identifier used to group related assets within a project. The meaning of the grouping is determined by the Importer. Photo Manager uses it when applying the group propagation policy.

Geographic Description (`geo_location`)

A geographic description associated with a photograph, determined by the Importer. It is distinct from the user-editable `location` field.

Geographic Location Key (`location_key`)

An identifier shared by photographs representing the same logical geographic location. Photo Manager uses it when applying the location propagation policy. Its format is defined by the Importer and is treated as an opaque value.

Image Folder (`image_folder`)

The `workspace` field that identifies the directory containing the project's photographs. Its value is relative to the directory containing `project.json`.

Propagation

The application of a metadata value to other photographs according to the selected metadata policy. The propagation scope may be the entire project (`event`), a group of photographs (`group`), photographs sharing a location key (`location`), or disabled (`none`).

Source

The external system or collection from which an Importer obtains photographs and associated information. The `workspace.source` and `workspace.source_id` fields identify the source and the particular source object associated with the project.

10.3. Developer Checklist

The following checklist provides a final review of an Importer before it is released.

It covers the principal requirements for producing a valid Photo Manager project and making it available to Photo Manager. It is not a substitute for the detailed requirements in this Handbook or the applicable Project JSON Reference.

10.3.1. Project Format and Compatibility

- The target Photo Manager version and applicable Project JSON Schema version have been identified.
- The Importer uses the correct version of the Project JSON Reference.
- The generated project contains all required top-level objects and fields.
- Field names, data types and permitted values conform to the specification.
- All required photo record fields are present, including fields for which no value is available.
- The `workspace.schema` value identifies the intended schema version.

Workspace and File Management

- The Importer obtains the configured Photo Manager workspace location rather than relying on a hard-coded path.
- The completed project is placed in its own directory directly beneath the configured workspace.
- The photograph directory is identified correctly by `workspace.image_folder`.
- Each photo record's `file` value identifies the correct photograph within that directory.
- File references use the required relative path conventions.
- Photograph filenames are unique within the project.
- The completed project contains all photograph files and supporting files required for normal use.

Acquisition and Metadata

- Each photograph recorded as successfully acquired has been stored in the project.
- Failed or unavailable photographs are excluded from the published project.
- Relevant source information and metadata have been preserved and represented in the appropriate fields.
- Metadata values have been interpreted carefully, without presenting uncertain information as established fact.
- Latitude and longitude are supplied as a pair, or both are null when unavailable.
- Metadata policies contain valid propagation values and reflect the intended initial behaviour.

Validation and Publication

- The completed project data can be read and parsed as valid JSON.

- The project has been checked against the Project JSON Reference.
- Every photo record refers to an existing, accessible photograph at the expected location.
- The project has passed its final integrity checks before publication.
- The project remains undiscoverable while it is incomplete or undergoing validation.
- Publication does not overwrite or modify an existing project.
- The Importer confirms that publication has completed successfully.
- The Importer does not continue modifying the project after publication.

Testing and Release

- The completed project has been discovered and opened successfully by the target Photo Manager version.
- Photographs display correctly and their associated metadata is available.
- Metadata editing and propagation behave as expected.
- Project file references remain valid when the complete project directory is moved.
- The Importer has been tested against relevant failure conditions, including interrupted acquisition and unsuccessful file operations.
- Errors and incomplete operations are reported clearly to the user.
- The Importer's documentation identifies its requirements, limitations and any relevant source-system restrictions.

An Importer that satisfies these checks and produces a project conforming to the applicable Project JSON Reference has met the principal requirements for integration with Photo Manager.

The final confirmation is successful use of the published project in Photo Manager.